SocialPilot

Social Media Scheduler & Campaign Management Platform

Project Plan — Sprint 1 Blueprint

Duration: 8 Weeks | Team: 5 Members

Stack: Python FastAPI · React/Next.js · PostgreSQL · MongoDB · Redis

1. Project Overview

SocialPilot is a centralized social media scheduling and campaign management SaaS platform. It enables content creators, digital marketing agencies, startups, and businesses to schedule, manage, and publish content across multiple social media platforms from a single unified dashboard.

Core Value Propositions

- Schedule and auto-publish content to Facebook, Instagram, LinkedIn, X (Twitter), YouTube, and Pinterest.
- Manage multi-platform campaigns with unified analytics and ROI tracking.
- Role-based team collaboration for agencies and marketing teams.
- Real-time engagement tracking, audience growth monitoring, and export reports.
- Notification workflows for publishing status, campaign alerts, and team activity.

Project Goals

#	Goal	Target
1	Scheduling success rate	≥ 95%
2	API response time	< 300 ms
3	Multi-platform publish accuracy	≥ 98%
4	Dashboard load time	< 2 sec
5	Concurrent user support	50+ users
6	Report generation	< 5 sec

2. Team Structure & Responsibilities

The team of 5 is organized into three functional streams: Frontend (2), Backend (2), and Database (1). Each member has a primary ownership domain but collaborates across streams.

Role	ID	Primary Stack	Core Responsibilities
Frontend Developer 1 (UI/UX Lead)	FE-1	React.js, Next.js, Tailwind CSS	Dashboard, Scheduling Calendar, Content Creator UI, Responsive Design
Frontend Developer 2 (Analytics & Integrations)	FE-2	React.js, Chart.js, Recharts, JWT	Analytics Dashboards, Campaign Views, Notification UI, API Integration Layer
Backend Developer 1 (Auth & Core APIs)	BE-1	Python, FastAPI, SQLAlchemy, JWT	Authentication, User Mgmt, Social Account Integration, Role-based Access Control
Backend Developer 2 (Scheduling & Publishing)	BE-2	FastAPI, Celery, Redis, Social APIs	Post Scheduling Engine, Publishing Queue, Celery Workers, Campaign & Analytics APIs
Database Engineer (Data Layer Lead)	DB	PostgreSQL, MongoDB, Redis, Alembic	Schema Design, Migrations, Query Optimization, Redis Caching, Data Warehouse

Detailed Responsibilities

FE-1 — Frontend UI/UX Lead

- Design and implement the main dashboard layout and navigation system.
- Build the Publishing Calendar component with drag-and-drop scheduling.
- Develop the Create Post page (text, image, video, carousel, story types).
- Implement content preview across platform-specific formats.
- Ensure full responsiveness using Tailwind CSS (mobile-first).
- Collaborate with BE-1 for Auth flows (login, registration, role-gating).

FE-2 — Frontend Analytics & Integration

- Build all analytics dashboard components using Chart.js and Recharts.
- Develop Campaign Management UI (creation, tracking, performance views).
- Implement Notification center (bell icon, real-time alerts, email status).
- Handle JWT token management, refresh, and protected route guards.
- Integrate Social Media SDK OAuth flows for account connection UI.
- Build Reports page with PDF/Excel export triggers.

BE-1 — Backend Auth & Core APIs

- Implement JWT-based authentication (register, login, refresh, logout).
- Build RBAC middleware for Content Creator, Marketing Team, Business, Admin roles.
- Develop User Management and Team Management APIs.
- Build Social Account Management module (OAuth token storage, sync, permissions).
- Integrate Facebook Graph API and Instagram Graph API OAuth flows.
- Write unit tests for all auth and account endpoints.

BE-2 — Backend Scheduling & Publishing Engine

- Design and implement the Content Scheduling API (CRUD, recurring, drafts).
- Build the Celery-based asynchronous publishing task queue with Redis broker.
- Integrate LinkedIn API, X (Twitter) API, and YouTube Data API.
- Implement publishing status tracking (Scheduled → Published / Failed / Cancelled).
- Build Campaign Management APIs (creation, grouping, tracking, metrics).
- Develop Analytics and Engagement Tracking API endpoints.
- Implement failed post retry logic with exponential backoff.

DB — Database Engineer

- Design complete PostgreSQL schema: users, teams, accounts, posts, campaigns, analytics.
- Design MongoDB collections: content metadata, media files, publishing logs.
- Write and manage all Alembic migration files.
- Configure Redis for session caching, Celery queues, and rate-limit counters.
- Set up object storage integration (AWS S3 / Cloudinary) for media files.
- Optimize slow queries with proper indexing and query planning.
- Design analytics data warehouse schema for reporting aggregations.

3. System Architecture

Layer Overview

Layer	Component	Technology
Client	Web / Mobile Application	React.js / Next.js, Tailwind CSS
API Gateway	Auth, Routing, Rate Limiting, Logging	FastAPI middleware, JWT
Microservices	User, Account, Content, Scheduling, Publishing, Campaign, Analytics, Notification	FastAPI (8 service modules)
Task Queue	Background Publishing & Notifications	Celery + Redis
Data — Relational	Users, Campaigns, Scheduling, Auth	PostgreSQL + SQLAlchemy ORM
Data — Document	Content Metadata, Media, Logs	MongoDB
Data — Cache	Sessions, Queues, Rate Limits	Redis
Data — Object Storage	Images, Videos, Media Files	AWS S3 / Cloudinary
Infrastructure	Containerization, Orchestration, CI/CD, Monitoring	Docker, Kubernetes, GitHub Actions, Prometheus/Grafana
External APIs	Social Platform Publishing	Facebook, Instagram, LinkedIn, X, YouTube, Pinterest APIs

API Design Conventions

- REST API with versioned endpoints: /api/v1/...
- JWT Bearer tokens in Authorization header for all protected routes.
- Consistent JSON response envelope: { success, data, message, errors }.
- Pagination via ?page=&limit= on all list endpoints.
- Background tasks (publishing, notifications) dispatched via Celery tasks.
- WebSocket channel for real-time notification delivery to frontend.

Database Schema — Key Tables

Database	Table / Collection	Key Fields
PostgreSQL	users	id, email, password_hash, role, team_id, created_at
PostgreSQL	social_accounts	id, user_id, platform, access_token, refresh_token, expires_at
PostgreSQL	posts	id, user_id, content, media_urls, platform, status, scheduled_at
PostgreSQL	campaigns	id, user_id, name, platform, start_date, end_date, budget, objectives
PostgreSQL	publishing_logs	id, post_id, status, platform_response, retried_at, error
MongoDB	content_metadata	post_id, hashtags, mentions, content_type, media_info

Database	Table / Collection	Key Fields
MongoDB	analytics_events	post_id, platform, impressions, reach, clicks, engagements, timestamp
Redis	session:{user_id}	JWT refresh token, TTL 7 days
Redis	queue:publish	Celery task payloads for scheduled posts

4. Day-Wise Tasks — Week 1 & 2

Week 1–2 covers Milestone 1: Project Initialization, System Design & Core Setup. All departments work in parallel with daily sync points.

Week 1 — Foundation Sprint

Day 1 — Kickoff & Architecture

Who	Tasks for Day 1
FE-1	Setup Next.js project with Tailwind CSS; configure folder structure (pages, components, hooks, services); push to GitHub.
FE-2	Install Chart.js and Recharts; create global theme (colors, fonts, spacing tokens); setup .env for API base URL.
BE-1	Initialize FastAPI project; setup project folder structure (routers, models, schemas, services, core); push to main.
BE-2	Setup Celery with Redis broker; create celery.py config; scaffold tasks/ folder; validate worker starts.
DB	Design full ER diagram for PostgreSQL; define all table names, fields, FK relationships; share draft with team for review.

Day 2 — Environment & Database

Who	Tasks for Day 2
FE-1	Create layout shell: Sidebar nav, Header, Main content area; implement route structure with Next.js app router.
FE-2	Setup Axios instance with baseURL and JWT interceptor; create auth context (useAuth hook).
BE-1	Configure PostgreSQL connection via SQLAlchemy; setup async engine; implement database.py session factory.
BE-2	Configure MongoDB connection using Motor (async); create MongoDB client singleton; test collection access.
DB	Write initial Alembic migration: create users, teams tables; run migration; validate schema in pgAdmin.

Day 3 — Authentication Core

Who	Tasks for Day 3
FE-1	Build Login page UI; Registration page; form validation with react-hook-form; connect to auth context.
FE-2	Implement protected route HOC; setup JWT decode and role-check utility; redirect unauthorized users.
BE-1	Implement /api/v1/auth/register, /login, /refresh, /logout endpoints; bcrypt password hashing; JWT generation.
BE-2	Define Pydantic schemas for all auth models; write unit tests for auth endpoints using pytest.
DB	Write migration: add social_accounts, roles tables; seed default roles (admin, creator, marketing, business).

Day 4 — User & Profile Management

Who	Tasks for Day 4
FE-1	Build Profile Settings page (avatar upload, name, bio, password change).
FE-2	Build Team Management page (invite member, assign role, remove member).
BE-1	Build /api/v1/users CRUD; profile update endpoint; avatar upload to S3 endpoint.
BE-2	Build /api/v1/teams endpoints (create team, invite, remove, roles); RBAC middleware decorator.
DB	Add indexes on users.email and social_accounts.user_id; write migration for team_members join table.

Day 5 — Social Account Integration

Who	Tasks for Day 5
FE-1	Build 'Connect Accounts' page; display connected platforms with icons, status, and disconnect button.
FE-2	Implement OAuth popup flow: open platform auth URL, handle callback redirect, store token via API.
BE-1	Build OAuth initiation endpoints for Facebook and Instagram; token exchange and storage logic.
BE-2	Build OAuth flows for LinkedIn and X (Twitter); validate tokens; implement token refresh scheduler.
DB	Encrypt access_token and refresh_token columns using pgcrypto; add token_expires_at field; update migration.

Day 6 — Dashboard Shell & Integration

Who	Tasks for Day 6
FE-1	Build main Dashboard page: summary cards (scheduled posts, active campaigns, followers); connect to API.
FE-2	Implement notification bell component; setup WebSocket connection for real-time alerts.
BE-1	Build /api/v1/dashboard/summary endpoint (aggregated counts); implement WebSocket notification broadcaster.
BE-2	Write integration tests for all auth + account flows end-to-end; fix any discovered bugs.
DB	Setup Redis session store; implement token blacklist for logout; test session TTL.

Day 7 — Buffer, Review & PR Merges

Who	Tasks for Day 7
FE-1	Code review of FE-2 PRs; fix UI inconsistencies; accessibility audit (ARIA labels, keyboard nav).
FE-2	Code review of FE-1 PRs; polish responsive behavior on mobile breakpoints.
BE-1	Code review of BE-2 PRs; API documentation in OpenAPI/Swagger; fix any auth edge cases.
BE-2	Code review of BE-1 PRs; validate all Celery tasks run correctly; check retry configs.
DB	Full DB audit: verify all FK constraints, indexes; run EXPLAIN ANALYZE on key queries; document schema.

Week 2 — Content & Publishing Engine

Day 8 — Content Creation APIs & UI

Who	Tasks for Day 8
FE-1	Build 'Create Post' page: rich text editor, platform selector checkboxes, character counter per platform.
FE-2	Implement media upload component (drag-and-drop, preview, multi-file, type validation).
BE-1	Build /api/v1/posts CRUD (create, read, update, delete, draft save) with media_urls field.
BE-2	Build media upload endpoint: receive file → upload to S3/Cloudinary → return URL; validate file types/sizes.
DB	Write posts table migration; add media_metadata collection in MongoDB; create content_drafts table.

Day 9 — Scheduling System

Who	Tasks for Day 9
FE-1	Build Publishing Calendar (monthly/weekly view) using react-big-calendar or FullCalendar; show scheduled posts.
FE-2	Build Schedule Post modal: date-time picker, timezone selector, recurring options (daily/weekly/monthly).
BE-1	Build /api/v1/posts/schedule endpoint; validate scheduled_at is future; store scheduling metadata.
BE-2	Implement Celery Beat scheduler: poll for due posts every minute; dispatch publish tasks to queue.
DB	Add scheduled_at, timezone, recurrence_rule columns to posts table; index on status + scheduled_at.

Day 10 — Publishing Engine

Who	Tasks for Day 10
FE-1	Build Publishing Queue page (list view with status badges: Scheduled / Published / Failed / Pending).
FE-2	Implement real-time status update on queue page via polling or WebSocket.
BE-1	Build /api/v1/publishing/logs and /api/v1/publishing/status endpoints.
BE-2	Implement Celery publish tasks for Facebook and Instagram; handle API response parsing; update post status.
DB	Create publishing_logs table; add retry_count, last_error columns; write migration.

Day 11 — More Platform APIs & Retry Logic

Who	Tasks for Day 11
FE-1	Build content preview component: simulate how post will look per platform (Twitter card, Instagram square, LinkedIn).
FE-2	Add 'Retry Failed Post' button on queue page; implement bulk action (cancel multiple, reschedule).
BE-1	Implement LinkedIn and X (Twitter) publish Celery tasks; unify publish task interface.
BE-2	Implement exponential backoff retry logic for failed tasks (max 3 retries); send failure notification.

Who	Tasks for Day 11
DB	Create index on publishing_logs(post_id, status, created_at); optimize queue query performance.

Day 12 — Campaign Module

Who	Tasks for Day 12
FE-1	Build Campaign Creation form (name, platform, start/end date, budget, objectives, KPIs).
FE-2	Build Campaign List and Campaign Detail pages with performance summary cards.
BE-1	Build /api/v1/campaigns CRUD; link posts to campaigns via campaign_id FK.
BE-2	Build /api/v1/campaigns/{id}/analytics endpoint; aggregate engagement metrics from analytics_events.
DB	Create campaigns and campaign_posts join table; write migrations; add composite indexes.

Day 13 — Analytics & Reporting

Who	Tasks for Day 13
FE-1	Build Analytics Dashboard: line charts (engagement over time), bar charts (platform comparison), KPI cards.
FE-2	Build Audience Analytics section: follower growth line chart, demographics pie chart, geographic map widget.
BE-1	Build /api/v1/analytics/content endpoint; support date range filter and platform filter query params.
BE-2	Build /api/v1/analytics/audience; implement data aggregation pipeline from MongoDB analytics_events.
DB	Design analytics_summary table for pre-aggregated daily rollups; write rollup Celery task (runs nightly).

Day 14 — Testing, Buffer & Week 2 Wrap

Who	Tasks for Day 14
FE-1	End-to-end testing of Schedule Post → Calendar → Queue flow; fix any UI bugs discovered.
FE-2	Test analytics dashboard with mock data; ensure chart responsiveness; fix tooltip formatting.
BE-1	Write pytest integration tests for Campaign and Publishing endpoints; achieve 70%+ coverage.
BE-2	Load test Celery publishing queue with 20 concurrent tasks; document throughput and failure rates.
DB	Full schema review; validate all migrations run cleanly from scratch; document DB README.

5. Git Workflow

Branch Strategy

The team follows a trunk-based development model with short-lived feature branches. The main branch is always deployable.

Branch	Owner	Purpose
main	All (protected)	Production-ready code. Merged via reviewed PRs only.
develop	All	Integration branch. All features merge here first.
feature/fe1-*	FE-1	UI/UX features (e.g. feature/fe1-dashboard-layout)
feature/fe2-*	FE-2	Analytics & integration features
feature/be1-*	BE-1	Auth & core API features
feature/be2-*	BE-2	Scheduling & publishing features
feature/db-*	DB	Schema, migrations, optimization work
hotfix/*	Any	Emergency fixes to main (fast-tracked PR review)

Commit Message Convention

`Format: <type>(<scope>): <short description>`

Type	Use For
feat	New feature or endpoint
fix	Bug fix
docs	Documentation changes
style	Formatting, no logic change
refactor	Code restructure, no feature change
test	Adding or updating tests
chore	Build scripts, CI configs, deps
db	Schema changes or migrations

Examples: feat(auth): implement JWT refresh endpoint | fix(scheduler): retry task on 429 rate limit | db(posts): add scheduled_at index

PR & Review Rules

- Minimum 1 reviewer required before merging to develop; 2 reviewers for main.
- No PR should exceed 400 lines changed — break large features into smaller PRs.
- All CI checks (tests, linting) must pass before merge.
- Reviewer must leave at least 2 specific comments (approval or requested changes).

- PRs must be merged within 24 hours of approval or re-reviewed.
- Delete branch after merge to keep repo clean.

CI/CD Pipeline (GitHub Actions)

- On every push: run linter (ESLint for FE, flake8 for BE) and unit tests.
- On PR to develop: run full test suite + build Docker image (dry run).
- On merge to main: build production Docker images, push to registry, deploy to staging.
- Secrets managed via GitHub repository secrets (never in code).

6. Risk Register

Risk	Impact	Probability	Owner	Mitigation
Social API rate limits (Twitter, Instagram) throttle publishing tasks	**High**	High	BE-2	Implement exponential backoff, respect rate limit headers, queue with delay, cache token scopes.
OAuth token expiry causes silent publish failures	**High**	Medium	BE-1 + DB	Store token expiry; Celery Beat job refreshes tokens 1hr before expiry; alert user if refresh fails.
Celery worker crashes under load	**Medium**	Medium	BE-2	Configure worker auto-restart; set task visibility timeout; monitor with Flower dashboard.
Database schema changes break existing features	**High**	Medium	DB	All changes via Alembic migrations only; test rollback; staging environment mirrors production schema.
Frontend-Backend API contract mismatch	**Medium**	High	FE-2 + BE-1	Define OpenAPI spec first; FE consumes Swagger docs; mock API (MSW) used during development.
Media upload size causing timeouts	**Medium**	Medium	BE-2 + DB	Frontend enforces 50MB limit; backend streams to S3 directly (presigned URL); async upload confirmation.
Team member blocked by unfamiliar tech	**Low**	Medium	All	Daily standups to surface blockers early; pair programming sessions; shared internal wiki.
Git merge conflicts on shared config files	**Low**	High	All	Dedicated team member owns .env.example and docker-compose.yml; changes go through PR with FYI tag.

7. Week 1 Deliverables

By end of Day 7, the following must be complete and demonstrable:

Frontend Deliverables

Deliverable	Owner	Acceptance Criteria
Next.js project initialized with Tailwind CSS	FE-1	Builds without error; folder structure matches agreed template.
Login & Registration pages	FE-1	Forms validate; JWT stored in httpOnly cookie; redirect to dashboard on success.
Dashboard shell with sidebar and header	FE-1	Responsive at 375px–1440px; active route highlighted in nav.
Protected routing & JWT interceptor	FE-2	Unauthenticated user redirected to /login; 401 response triggers refresh attempt.
Connect Accounts page (OAuth flow)	FE-2	Platform OAuth popup opens; success updates account status in UI.
Notification bell component	FE-2	Renders; WebSocket connection established (even if no data yet).

Backend Deliverables

Deliverable	Owner	Acceptance Criteria
FastAPI project with folder structure	BE-1	Starts with uvicorn; /api/v1/health returns 200.
JWT Auth endpoints (register, login, refresh, logout)	BE-1	Postman tests pass; invalid credentials return 401; tokens expire correctly.
RBAC middleware	BE-1	Admin-only endpoints reject non-admin; role escalation not possible.
User Profile & Team Management APIs	BE-1	CRUD endpoints tested; pagination works.
Social Account OAuth (Facebook, Instagram)	BE-1	Token stored encrypted; account appears in /api/v1/accounts.
Social Account OAuth (LinkedIn, X/Twitter)	BE-2	Tokens stored; platform-specific scopes validated.
Celery + Redis setup	BE-2	Worker starts; test task executes and returns result via flower.
Dashboard summary endpoint	BE-1	Returns aggregated counts; response < 300ms on local.

Database Deliverables

Deliverable	Owner	Acceptance Criteria
Finalized ER diagram shared with team	DB	All tables documented; team signed off; no orphan FK references.
Alembic migrations: users, teams, social_accounts	DB	alembic upgrade head runs cleanly; alembic downgrade head reverses cleanly.
Redis session store configured	DB	JWT blacklist stores on logout; TTL-based expiry verified.
Encrypted token columns	DB	access_token stored encrypted; plaintext not visible in DB dump.
Schema documentation README	DB	README.md in /database folder explains all tables, relationships, and indexes.

8. Week 2 Preparation

Before starting Day 8, every team member must complete the following preparation checklist. Week 2 moves faster — these prerequisites ensure zero ramp-up time.

Pre-Week 2 Checklist

#	Task	Owner	Deadline
1	All Week 1 PRs merged to develop; no open review comments.	All	Day 7 EOD
2	OpenAPI spec for Week 2 endpoints (posts, scheduling, publishing, campaigns) drafted and shared.	BE-1 + BE-2	Day 7 EOD
3	Mock API responses added to MSW (for FE to work independently if BE not ready).	FE-2	Day 7 EOD
4	AWS S3 bucket (or Cloudinary account) created; credentials in .env.example; access tested.	DB + BE-2	Day 7 EOD
5	react-big-calendar (or FullCalendar) installed and a test calendar page renders.	FE-1	Day 7 EOD
6	Celery Beat configured with 60-second interval test task; confirmed running in Docker Compose.	BE-2	Day 7 EOD
7	Social API test credentials for all 4 platforms (FB, IG, LinkedIn, X) obtained and stored in .env.	BE-1	Day 7 EOD
8	Database migrations up to Week 1 schema tested on fresh DB (drop + recreate + upgrade head).	DB	Day 7 EOD

Week 2 Focus Areas by Role

Role	Week 2 Primary Focus	Key Outcomes
FE-1	Publishing Calendar, Create Post UI, Content Preview	User can create, preview, and schedule a post on the calendar.
FE-2	Queue Page, Campaign UI, Analytics Dashboard	User can see publishing queue status and campaign performance.
BE-1	Post CRUD APIs, Campaign APIs, Analytics Endpoints	All content and campaign endpoints live with proper validation.
BE-2	Celery Publishing Tasks (all 4 platforms), Retry Logic	Posts auto-publish at scheduled time; failures retry with backoff.
DB	Posts/Campaign Schema, Publishing Logs, Analytics Rollups	Analytics data pipeline producing daily summaries in < 5 sec.

Definition of Done — Week 2

- A user can create a text + image post, schedule it for a future time, and see it on the calendar.
- Celery worker picks up the scheduled post and publishes it to at least 2 live social platforms.
- Publishing queue page shows real-time status update (Scheduled → Published or Failed).
- Campaign creation flow works end-to-end with posts linked to a campaign.
- Analytics dashboard renders engagement charts for at least one platform's mock or live data.
- All new endpoints covered by at least basic integration tests.